GM ECU Simulator

User Manual

J2534 v04.04 PassThru Device Emulator
for GMLAN / GMW3110-2010

Version 1.1 — May 2026

1. Introduction

GM ECU Simulator is a Windows desktop application that emulates one or more GM (GMLAN / GMW3110) Engine Control Units and registers itself as a real SAE J2534-1 v04.04 PassThru device. Any J2534-aware diagnostic host — Tech 2 Win, GDS, or a custom scan tool — discovers it through the standard Windows registry path and connects to it exactly as it would connect to a physical hardware interface such as the Tactrix OpenPort or MongoosePro.

PID values are synthesised in real time from configurable waveforms (sine, triangle, square, sawtooth, constant, or file-stream replay). The waveform parameters are editable live, without restarting.

1.1 Supported Diagnostic Services

Service	Name	Description
$22	Read Data by Identifier	Reads one or more PIDs synthesised from waveforms
$2C	Dyn. Define Data ID	Defines custom DPIDs for periodic UUDT streaming
$2D	Write Memory by Address	32-bit memory-address writes mirrored to matching PID
$AA	Periodic UUDT Push	Streams DPID values at Slow / Medium / Fast rates
$27	Security Access	Modular seed/key unlock — see §8
$3E	Tester Present	Keeps the session alive; resets P3C timeout
$10	Start Diagnostic Session	Transitions ECU into programming or extended mode
$20	Return to Normal	Graceful session teardown and DPID scheduler reset

2. Architecture

The simulator is split into two components that communicate over a Windows named pipe (\\.\pipe\GmEcuSim.PassThru).

2.1 PassThruShim (native DLL)

A thin C++ DLL built for both x64 and x86. Registered in the Windows registry as a J2534 v04.04 device under HKLM\SOFTWARE\PassThruSupport.04.04\GmEcuSim and the WOW6432Node mirror. When a diagnostic host loads it with LoadLibrary, it opens the named pipe and forwards all 14 PassThru API calls as length-prefixed binary frames. The shim exports exactly the 14 functions mandated by SAE J2534-1 v04.04 — no v05.00 extensions, no Drew Tech proprietary exports.

2.2 GmEcuSimulator.exe (WPF application)

A WPF .NET 9 application. On startup it opens the named pipe server and begins accepting connections. The RequestDispatcher maps each incoming IPC message type to VirtualBus operations. The VirtualBus routes incoming CAN frames by destination ID to one of N EcuNode instances. Each EcuNode runs ISO-TP reassembly on inbound frames and dispatches the assembled USDT payload to the matching service handler. Responses are enqueued back onto the channel RX queue for the shim to drain via PassThruReadMsgs.

2.3 Both bitnesses are required

A Windows process can only load a DLL matching its own bitness. 64-bit hosts (e.g. the DataLogger application) load PassThruShim64.dll; 32-bit hosts (Tech 2 Win, GDS, Bosch MDI) load PassThruShim32.dll. The OS automatically redirects 32-bit registry reads to WOW6432Node. Register.ps1 writes both registry views pointing at the matching DLL.

3. Getting Started

3.1 Prerequisites

- Windows 10 or Windows 11 (64-bit)
- Administrator rights (required for registry registration)
- .NET 9 Runtime (included in build output)
- Microsoft Visual C++ Redistributable 2022 (required by the native shim)

3.2 First Run

1. Build the solution with `dotnet build "GM ECU Simulator.sln"` and the native shim with MSBuild, or use pre-built binaries.
2. Launch GmEcuSimulator.exe. The named pipe server starts automatically and the WPF editor opens.
3. Click J2534 > Register as J2534 device.... A UAC elevation prompt appears; approve it to write the registry entries.
4. The status bar updates to Registered (32-bit + 64-bit).
5. Your J2534 diagnostic host now lists "GM ECU Simulator" in its device dropdown.

To undo registration, use J2534 > Unregister. To verify what is registered, use J2534 > Show registered devices....

4. Main Editor

The main editor is divided into three areas: the ECU list on the left, the PID table in the centre, and the waveform configuration panel on the right. A tab strip at the bottom provides access to Bus Log, Bin Replay, Security access ($27), and Glitch Settings.

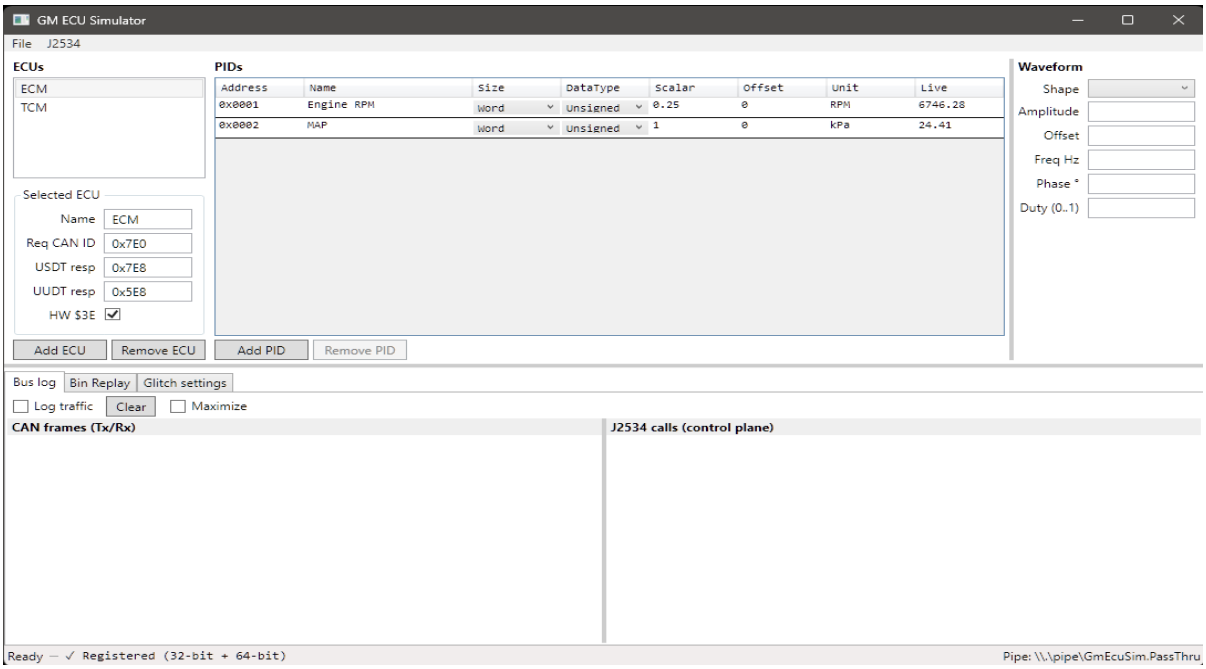

Figure 1 — Main editor with ECM and TCM in the ECU list, Engine RPM and MAP sensor PIDs, and the sine-wave waveform panel on the right.

4.1 ECU List

Lists all virtual ECU nodes loaded in the simulator. Each entry shows the ECU name, request CAN ID, and response CAN ID. Click an ECU to load its PIDs into the centre table. Use the toolbar buttons or Edit menu to add, remove, or clone ECUs.

4.2 PID Table

Lists all Data Identifiers (DIDs) configured for the selected ECU. Each row shows the two-byte PID identifier, a human-readable name, scaling formula, and the current synthesised value updating in real time. Click a row to load its waveform parameters into the right-hand panel.

4.3 Waveform Panel

Controls how the selected PID value evolves over time. The available shapes are:

- Sine — Smooth sinusoidal oscillation between (Offset - Amplitude) and (Offset + Amplitude).
- Triangle — Linear ramp up then ramp down.
- Square — Alternates instantly between two fixed levels.
- Sawtooth — Linear ramp up with instant reset.
- Constant — Fixed value equal to Offset; Amplitude and Frequency are ignored.
- File Stream — Replays values read sequentially from a CSV or binary log file.

All waveform parameter changes take effect immediately without restarting. The Frequency (Hz) field controls oscillation rate for time-varying shapes.

5. Bus Log

The Bus Log tab captures all J2534 API calls made by connected hosts and the resulting CAN frames on the virtual bus. Enable it by checking Log traffic in the tab header.

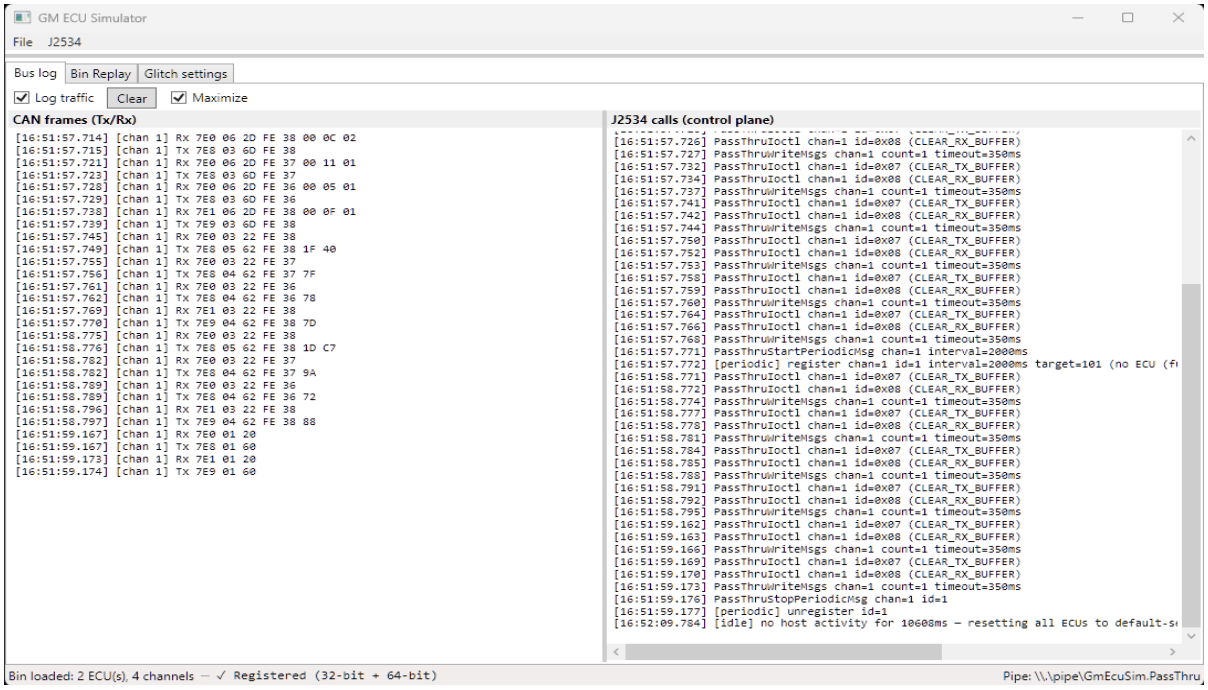

Figure 2 — Bus Log tab showing Rx $22 requests from the host at 0x7E0 and Tx $62 responses from ECM at 0x7E8, alongside the J2534 API call trace.

The left column shows raw ISO-TP / CAN frames with direction (Rx/Tx), CAN arbitration ID, and hex payload. The right column shows the corresponding J2534 API trace (PassThruOpen, PassThruConnect, PassThruWriteMsgs, PassThruReadMsgs, etc.) with return codes, confirming the host/simulator handshake.

Click Clear to wipe the log. The log is in-memory only and is not persisted to disk.

> **Warning — performance limitation:** Enable Log traffic only for short diagnostic bursts (under 10 seconds on high-speed runs). The text box does not virtualise its content and becomes quickly overwhelmed at high message rates, causing the GUI to freeze. This is a known optimisation issue slated for a future build.

6. Bin Replay

The Bin Replay tab lets you load a logged binary data file (typically recorded by the companion GM DataLogger application) and feed its channel values back through the simulator as if they were live ECU responses. This enables repeatable testing of diagnostic software against a known data set.

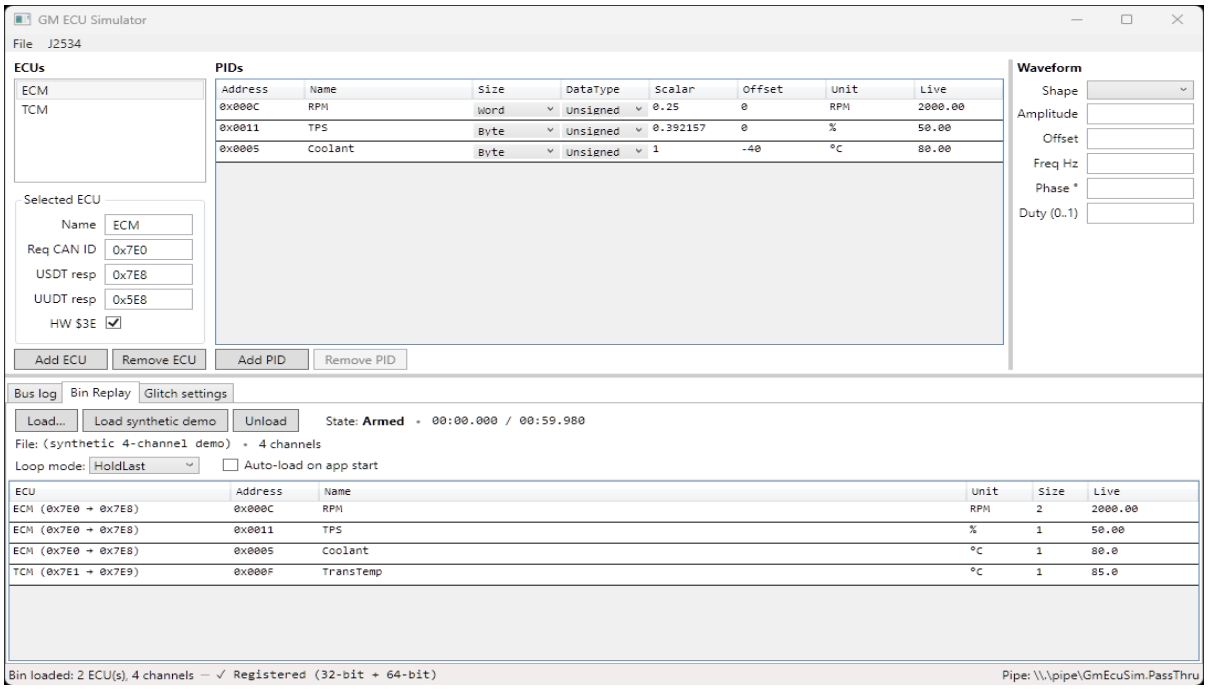

Figure 3 — Bin Replay tab with a four-channel demo file loaded. State: Armed. Channels: RPM, TPS, Coolant Temp (ECM) and Trans Temp (TCM).

- Load — Opens a file picker to select the .bin replay file.
- State — Shows the current playback state: Idle, Armed, or Playing.
- Channels — Lists data channels in the file with source ECU, PID ID, and unit.
- Arm / Disarm — Prepares the replay engine; playback starts on the next J2534 host connection.

While replay is active, waveform-synthesised values for matching PIDs are replaced by the file data. PIDs not present in the file continue to use their configured waveforms.

7. Glitch Settings

The Glitch Settings tab injects controlled faults into simulator responses. For each diagnostic service on each ECU you can set a fault probability (0-100%) and the action to take when the fault fires. Useful for testing how host software handles unexpected ECU behaviour.

Available glitch actions:

- EmitNrc — Respond with a Negative Response Code drawn from the NRC pool instead of the normal positive response.
- Drop — Silently discard the request; the host times out waiting.
- CorruptByte — Send the normal response with one byte flipped at random.
- Random — Choose one of the above actions randomly each time the fault fires.

The NRC Pool checkboxes select which NRC codes the EmitNrc action draws from: $11 (Service Not Supported), $12 (Sub-Function Not Supported), $22 (Conditions Not Correct), $31 (Request Out of Range), $33 (Security Access Denied), $78 (Busy Response Pending).

> **Note — not yet active:** The glitch injection UI is fully built and all settings are saved to the config file, but the feature currently has no effect at runtime. The bus logic does not yet consult the glitch configuration. Full implementation is slated for a future release.

8. Security Access ($27)

The Security access ($27) tab configures GMW3110 Service $27 SecurityAccess per ECU. The simulator implements $27 behind a two-layer plug-in interface so different GM seed-key flavours (the algorithms have evolved across model years) can be slotted in without recompiling the dispatcher.

8.1 Architecture in 30 seconds

- ISecurityAccessModule — owns one full $27 exchange step. The bundled Gmw3110_2010_Generic implementation handles all the protocol envelope concerns: length validation, subfunction parity (odd = requestSeed, even = sendKey), pending-seed tracking, the 3-strike attempt counter, the 10-second lockout window, and every NRC path ($12, $22, $35, $36, $37).
- ISeedKeyAlgorithm — the small strategy interface most users actually write. It supplies SeedLength, KeyLength, SupportedLevels, GenerateSeed and ComputeExpectedKey. Gmw3110_2010_Generic wraps an instance and handles everything else.
- SecurityModuleRegistry — maps a string ID (persisted in ecu_config.json) to a factory. Each ECU gets its own module instance, so per-ECU bookkeeping is isolated.

8.2 The wire exchange

A full successful unlock looks like this (ISO-TP Single Frames on the ECU's physical request and USDT response CAN IDs):

```
Tester → ECU   02 27 01                  // requestSeed level 1
ECU   → Tester 04 67 01 SS SS            // seed bytes
Tester → ECU   04 27 02 KK KK            // sendKey
ECU   → Tester 02 67 02                  // positive — unlocked
```

Failed unlock paths:

- Invalid key — NRC $7F 27 35. Attempt counter increments.
- Third failure — NRC $7F 27 36 and the ECU arms a 10-second lockout deadline.
- Any $27 request inside the lockout window — NRC $7F 27 37. Deadline self-heals; the next request after expiry is processed normally with the counter reset.
- sendKey without a prior requestSeed for the same level — NRC $7F 27 22.
- Malformed payload, reserved subfunction ($00 / $7F), or unsupported level — NRC $7F 27 12.

8.3 Configuring per ECU

In the editor, select the ECU you want to configure, then open the Security ($27) tab. The Module dropdown lists every algorithm registered in SecurityModuleRegistry, prefixed with a

synthetic "(none)" entry. Picking (none) leaves $27 unsupported on that ECU; the simulator returns NRC $11 ServiceNotSupported to any incoming request.

Beneath the dropdown is a key/value editor. Each row becomes one top-level property in the JSON object passed to the algorithm's LoadConfig. Values persist as JSON strings; the algorithm is responsible for parsing them as needed (hex bytes, integers, etc.). The grid round-trips to the SecurityModuleConfig entry under that ECU in ecu_config.json.

Example — with Module set to gm-e38-test, one row:

```
Key: fixedSeed       Value: 1234
```

…fixes the seed to 0x1234 for every $27 $01 on that ECU, which is useful for deterministic unit tests and for stepping through a tester's handler code without chasing a moving target. The corresponding key the tester must send back is E38(0x1234) = 0x96CE.

Multiple rows — each becomes one property in a flat JSON object. Algorithms that take more than one tunable read each by key name; algorithms that take fewer simply ignore the extras. Example with hypothetical extra metadata:

```
Key: fixedSeed       Value: 1234
Key: notes           Value: matches VIN ABC123
```

would serialise to { "fixedSeed": "1234", "notes": "matches VIN ABC123" }. The gm-e38-test algorithm reads fixedSeed and silently ignores notes — the row round-trips to disk but has no effect on the exchange.

> **Common point of confusion:** fixedSeed is optional. With the K/V grid completely empty, gm-e38-test still works — it just generates a random seed per request and validates whatever key the tester computes from it’s client-side. Both sides only need to agree on the algorithm; the config tunes behaviour, it does not gate it.

- Empty key, value-only — that row is skipped on serialise.
- Duplicate key on two rows — the row evaluated last wins (Dictionary semantics).
- Key the module does not recognise — silently ignored. No error surface for typos. Verify the algorithm consumed your config by triggering an exchange and watching the wire.

8.4 Live state and Reset

The strip above the K/V grid displays the selected ECU's live security state, refreshed at 10 Hz from the main UI tick. It reflects runtime state, not config:

- Status — Locked / Unlocked (level N) / Locked out — X.X s remaining. The countdown ticks down in 0.1 s steps; once the deadline elapses the next $27 request resets the counter automatically.
- Failed attempts — N / 3. Increments on each NRC $35; rolls back to 0 on a successful unlock or after the lockout deadline passes.
- Pending seed — the level and bytes of the last requestSeed response, or "(none)". Cleared on a successful unlock or when the lockout arms.

Per GMW3110 spec, the unlocked level persists across $20 ReturnToNormalMode, P3C TesterPresent timeouts, and host disconnects (PassThruDisconnect). Only a power-cycle clears it — and the simulator treats its own process restart as the equivalent of a power-cycle. So

opening the simulator and seeing "Unlocked" with an empty K/V grid is normal if a prior tester session already unlocked it.

The Reset state button next to the Module dropdown re-locks the ECU and clears the attempt counter, pending seed, and lockout deadline in one click — without restarting the simulator. Equivalent to a power-cycle for the $27 subsystem of that one ECU. Useful for iterative testing: verify lockout-then-recovery without waiting 10 s, retry a failed unlock cleanly, etc.

8.5 Bundled algorithms

The simulator ships with two algorithms registered out of the box:

gmw3110-2010-not-implemented

A placeholder stub. Returns the deterministic seed [0x12, 0x34] and refuses every key, regardless of what was sent. The intent is to exercise every NRC path end-to-end (requestSeed → 67 01 12 34, sendKey → 7F 27 35, three failures → 7F 27 36, lockout window → 7F 27 37, recovery) without committing any real algorithm math to the source tree.

gm-e38-test

The GM E38 ECM algorithm (also known as GMLAN algorithm 0x92; the same algorithm is used by the E67 ECM). 2-byte seed, 2-byte key, level 1 only. Random seed per request by default; supply a fixedSeed config entry for deterministic exchanges. The algorithm is documented openly in the GM tuning community; the simulator implementation is verified against five test vectors in Tests.Unit/Security/E38AlgorithmTests.cs.

> **On responsible use:** These algorithms are included as development aids for testing J2534 hosts against the simulator. They are not intended as tools for bypassing security on a vehicle you do not own or have rights to.

8.6 Writing a custom module

Adding a new algorithm is approximately 30 lines of C# plus a one-line registry update. The typical shape:

6. Implement ISeedKeyAlgorithm in Core/Security/Algorithms/. Declare SeedLength, KeyLength, SupportedLevels, then provide GenerateSeed (writes seed bytes into a buffer) and ComputeExpectedKey (writes the key the tester is expected to send back). Optionally accept configuration via LoadConfig(JsonElement?).
7. Register in SecurityModuleRegistry's static constructor: Register("my-algo", () => new Gmw3110_2010_Generic(new MyAlgorithm(), id: "my-algo")).
8. Add unit tests under Tests.Unit/Security/. Document at least one (seed, expected key) test vector in source. If you have a captured pair from real hardware, prefer that to a self-derived vector.

For algorithms whose protocol flow does not fit the standard requestSeed / sendKey envelope (multi-message exchanges, MAC-based schemes), implement ISecurityAccessModule directly and bypass Gmw3110_2010_Generic. See docs/security-access-modules.md for the full walkthrough using the E38 algorithm as a worked example.

9. J2534 Menu

The J2534 menu manages the simulator's registration as a PassThru device. Register and Unregister require administrator rights; a UAC elevation prompt appears automatically.

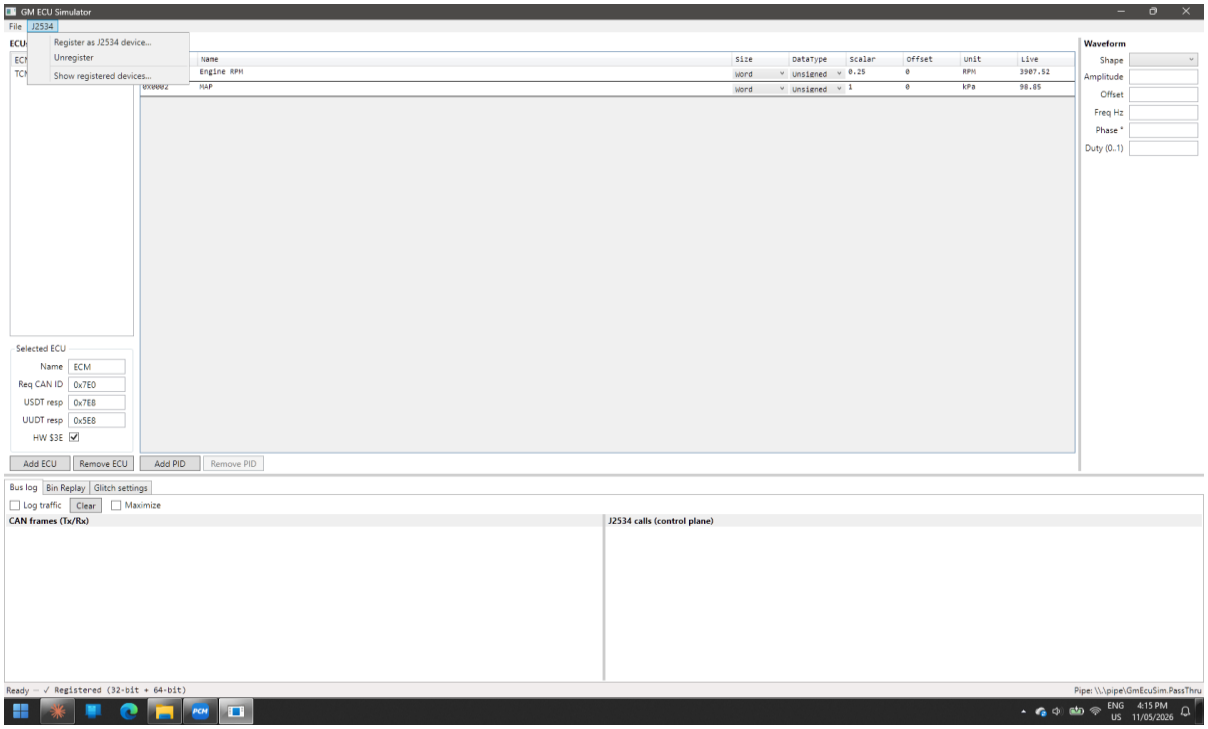

Figure 4 — J2534 menu with registration management commands.

Register as J2534 device... — Writes registry entries under HKLM\SOFTWARE\PassThruSupport.04.04\GmEcuSim (64-bit) and WOW6432Node (32-bit), each pointing at the matching PassThruShim DLL. Re-running Register over an existing entry automatically cleans up legacy layouts.

Unregister — Removes the registry entries. The simulator continues to accept pipe connections from direct clients; only J2534 host enumeration is affected.

Show registered devices... — Opens a read-only modal listing every J2534 device found on this machine across both registry views, with DLL existence checks. Run this before and after Register / Unregister to verify what changed.

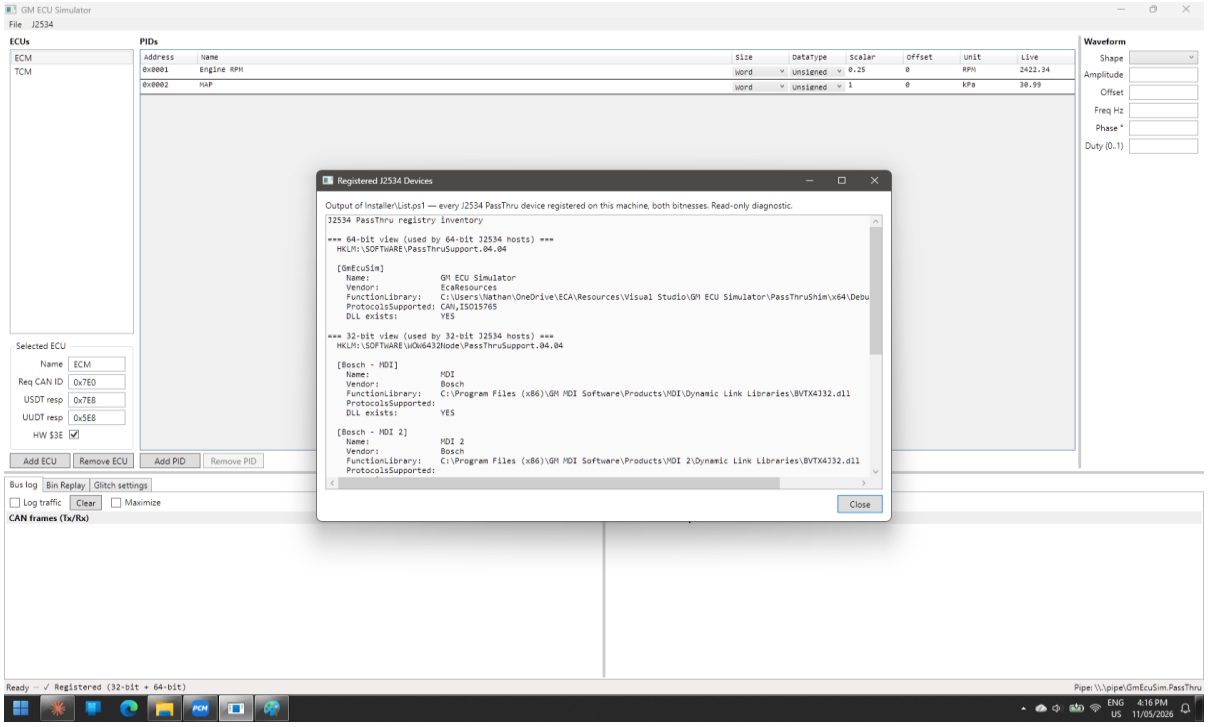

Figure 5 — Show Registered Devices dialog listing all J2534 devices found in the Windows registry with DLL path and existence status.

10. File Menu

The File menu manages ECU configuration files. The default configuration is ecu_config.json in the application directory and is loaded automatically on startup. Configurations written by v1.1+ are schema version 3; v1/v2 files load with security module fields null and $27 returns NRC $11 as before.

New (Ctrl+N) — Creates a blank configuration with a single default ECU.

Open... (Ctrl+O) — Loads a configuration from a JSON file.

Save (Ctrl+S) — Saves to the current file, or prompts for a path if unsaved.

Save As... (Ctrl+Shift+S) — Saves to a new file path.

Import... — Imports ECU definitions from an external format.

Export... — Exports the configuration to a distributable format.

Exit — Closes the application; the named pipe server shuts down gracefully.

11. Technical Reference

11.1 CAN ID Conventions

The simulator uses the OBD-II GM GMLAN CAN ID scheme matching real hardware. The GMW3110 specification uses pedagogical IDs ($241/$641) in its worked examples; those IDs are used only in unit-test fixtures and do not appear in the default configuration.

ECU	Request ID	Response ID	Functional ID
ECM	0x7E0	0x7E8	0x5E8
TCM	0x7E1	0x7E9	0x5E9

11.2 J2534 Conformance

PassThruShim exports exactly the 14 functions defined by SAE J2534-1 v04.04: PassThruOpen, PassThruClose, PassThruConnect, PassThruDisconnect, PassThruReadMsgs, PassThruWriteMsgs, PassThruStartPeriodicMsg, PassThruStopPeriodicMsg, PassThruStartMsgFilter, PassThruStopMsgFilter, PassThruSetProgrammingVoltage, PassThruReadVersion, PassThruGetLastError, and PassThruIoctl. PassThruReadVersion reports firmware version 04.04. The v05.00 functions (ScanForDevices, GetNextDevice, LogicalConnect, etc.) and the Drew Tech proprietary PassThruGetNextCarDAQ are deliberately not exported.

11.3 Supported Protocol — CAN only

PassThruConnect accepts ProtocolID.CAN (5) only. Hosts that attempt to connect with ProtocolID.ISO15765 (6) or any other protocol receive ERR_INVALID_PROTOCOL_ID from PassThruConnect. The simulator also surfaces the rejection in two places so a third-party user can see why their host failed to connect: a diagnostic entry in the J2534 calls pane ("[connect] rejected: protocol ISO15765 not supported — sim is CAN-only") and a status-bar message at the bottom of the main window.

CAN-only is a deliberate scope decision, not a bug. Hosts must therefore do their own ISO-TP framing in the PassThruWriteMsgs payload — prepending the PCI byte for Single Frames, splitting larger USDT messages into First Frame + Consecutive Frames, and handling Flow Control. On a real GM ECU, ProtocolID.ISO15765 mode (where the J2534 driver handles ISO-TP for the host) is the more common choice; against the simulator, point your host at ProtocolID.CAN and supply the PCI yourself.

11.4 Named Pipe Protocol

The shim communicates with the simulator over the named pipe \\.\pipe\GmEcuSim.PassThru using length-prefixed binary frames. Each frame starts with a 4-byte little-endian length followed by a message-type byte and the marshalled parameters. The RequestDispatcher maps each message type to the corresponding VirtualBus operation and enqueues the response.

11.5 ISO-TP Transport

Inbound CAN frames are processed by the ISO-TP reassembler, which handles Single Frame (SF), First Frame + Consecutive Frame (FF/CF), and Flow Control (FC) types per ISO 15765-2. Assembled USDT payloads are dispatched to the appropriate GMW3110 service handler by service ID (first payload byte).

11.6 Tester Present and Session Timeout

Each ECU tracks its own P3C (Tester Present) timer. If no $3E request arrives within 5 seconds (P3Cnom), the ECU runs EcuExitLogic: the DPID scheduler is cleared, $2D-defined dynamic PIDs are pruned, and an unsolicited $60 response is emitted. The same exit logic fires on

graceful $20 Return-to-Normal and on host-vanish detection (IdleBusSupervisor monitors inactivity on the VirtualBus). $27 SecurityAccess state (unlocked level, failed-attempt counter, lockout deadline) is retained across $20 per the GMW3110 specification — only a power-cycle (simulator restart) resets it.

11.7 NodeState

All per-ECU runtime state lives in a unified NodeState container exposed as node.State on EcuNode. This includes the ISO-TP reassembler, the dynamic DPID map, the $2D dynamic-PID set, the P3C TesterPresent state, the last enhanced channel, and the security-access fields (unlocked level, pending seed, failed attempts, lockout deadline, opaque module-state slot). Future services consult node.State for preconditions like "ECU must be unlocked at level N" by reading the same object that $27 writes.